计算机组成原理实验课程第 6—7 次实验报告

实验名称	五级流水线 CPU 指令扩展与验证			班级	0965
学生姓名	甘文杰	学号	2412700	指导老师	董前琨
实验地点	实验楼 A 304		实验时间	2026 年 10 月 22 日	

实验目的

本实验在已有五级流水线 CPU 和 Lab3/Lab4 数据前递框架上，完成 Lab6 与 Lab7 的累计功能扩展。Lab6 主要加入算术逻辑指令、乘除法及 HI/LO 特殊寄存器操作；Lab7 继续加入控制转移、延迟槽和字节/半字/非对齐访存指令。实现时需要维持流水级握手、同步 RAM 时序和精确的写回边界，避免多周期运算期间重复接受或提交指令。

本次验收目标包括：按课程测试程序逐点检查 Lab6、Lab7；使用独立 CPU132 参考模型生成退休 trace 并与 DUT 比对；通过 HI/LO 单元级测试和除法期间分支等待定向用例补充检查；记录仿真周期、停顿、旁路使用和零号寄存器写回情况。

实验环境与总体设计

实验使用 Vivado 2025.2、XSim 和 Windows 普通 PowerShell。Lab3 至 Lab7 共用一个 Vivado 工程，顶层为 `soc_lite_top`，仿真顶层为 `tb_top`，目标器件为 `xc7a200tfbg676-2`。CPU 数据通路由 IF、ID、EX、MEM、WB 五级构成；IF 使用同步 ROM，MEM 连接 32 位数据 RAM。验证脚本在 Vivado 启动前检查执行环境，并为每轮仿真保留编译、展开、仿真阶段日志和结构化 `result.txt`。

图 1 是根据 RTL 接口和流水级职责绘制的结构示意图。图中前递回路、多周期 HI/LO 等待以及 MEM 字节通道用于解释设计关系；它是 Python 生成的说明图，不是 Vivado 仿真截图。完整流水级信号与模块边界仍以工程 RTL 为准。

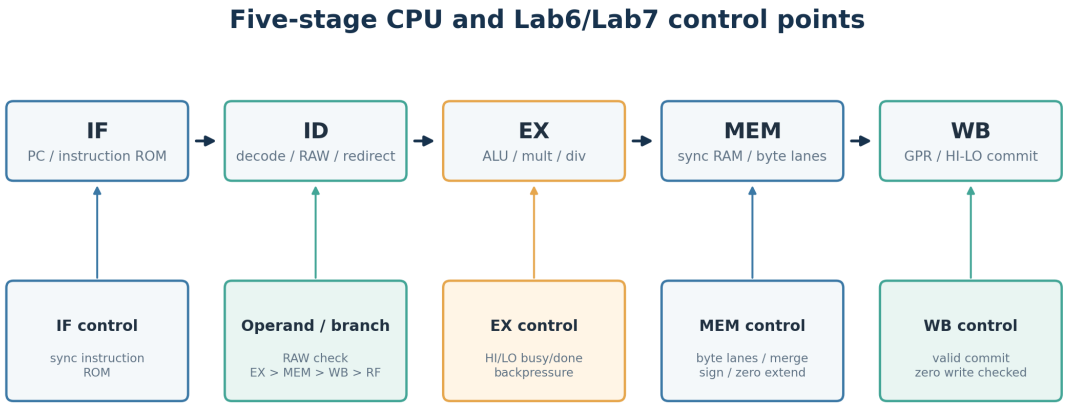


图 1: 五级流水线及 Lab6/Lab7 主要控制点（Python 绘制的结构示意图）

Lab6：算术逻辑与 HI/LO 扩展

算术逻辑指令。 ID 阶段增加 `add/sub/addi/slti/sltiu/andi/ori/xori/sllv/srav/srlv`

的译码，并相应确定源寄存器使用、目的寄存器和合法写回条件。EX 阶段对 `andi/ori/xori` 使用零扩展立即数，其余立即数按指令语义符号扩展；有符号比较与无符号比较分开处理。可变移位由 `rs[4:0]` 提供移位量、`rt` 提供被移位数据。

为避免未知或不支持编码造成伪写回，写回资格来自显式的合法指令集合，而不是仅凭某个译码位推断。新增 65-word 算术逻辑定向映像覆盖立即数边界、有符号/无符号比较、三种可变移位、高位移位量忽略、流水前递和 load-use。该定向 DUT 测试通过，73 周期、1 次 load-use 停顿，EX/MEM/WB 前递次数为 7/6/3，且 trace 与独立参考一致。

HI/LO 运算。新增 `hilo_unit.v` 封装乘法和除法 IP。乘法支持 signed/unsigned；除法器为 unsigned IP，signed 运算通过输入幅值化和输出符号恢复实现。乘法结果写为 HI= 乘积高 32 位、LO= 乘积低 32 位；除法结果写为 HI= 余数、LO= 商。`mfhi/mflo` 在 EX 使用 HI/LO 当前值，`mthi/mtlo` 在指令能够离开 EX 的提交边界更新特殊寄存器。

多周期乘除法在 EX 保持当前指令，`busy` 期间阻止更年轻指令越过；完成结果保持到 `consume_done`，随后回到空闲态。`start` 仅在该指令有效、属于乘除法、且单元既不忙也未完成时发出。以下为实际 RTL 中关键握手的节选：

```
assign es_hilo_start = es_valid && es_hilo_multdiv &&
                       !es_hilo_busy && !es_hilo_done;
assign es_hilo_consume = es_valid && es_hilo_done && ms_allowin;
assign es_ready_go = !es_hilo_multdiv || es_hilo_done;
assign es_allowin = !es_valid || es_ready_go && ms_allowin;
assign es_to_ms_valid = es_valid && es_ready_go;
```

图 2 展示单元请求、等待、完成保持和消费的状态关系。该图是依照 RTL 独立 testbench 观察到的握手绘制，图中状态名与实现一致。

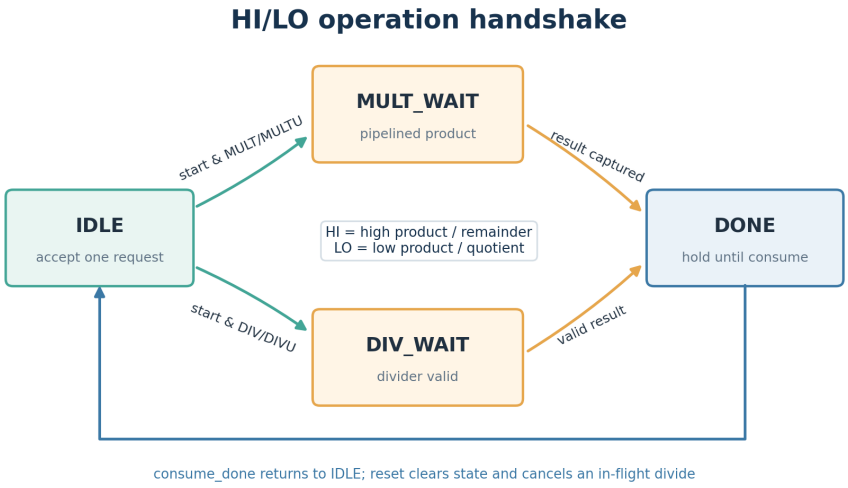


图 2: HI/LO 单元的请求与完成握手

HI/LO 单元测试。独立 testbench 共接受 3 次乘法和 4 次除法请求，完成并提交 6 次；另有一次在完成前由复位取消。检查覆盖 signed/unsigned 运算、零乘数、完成保持直到消费、单次接受与单次提交、`MTHI/MTLO` 以及复位清除进行中的除法。testbench 通过。除法除数为零不属于本次已记录的测试覆盖，报告不据此推断其行为。

Lab7：控制转移、延迟槽与访存

控制转移。新增 `j`、`jalr`、`bgez/bgtz/blez/bltz` 和 `bgezal/bltzal`。条件分支按有符号 `rs` 判断；`jalr` 的目标来自经过前递的 `rs`，链接目的寄存器使用指令 `rd`。条件分支目标按 `PC+4+signext(imm<<2)` 计算，跳转目标使用 `(PC+4)[31:28]` 与索引字段拼接，链接值为 `PC+8`。`bgezal/bltzal` 的链接行为按课程测试宏要求，即使条件不成立也写链接值，是否重定向仍由条件决定。

分支仅在 ID 指令能够被 EX 接收时发出，保留一条延迟槽指令；若 EX 被多周期运算占用，ID 中的分支保持等待，不重复重定向。该路径使用独立定向程序验证：`DIV` 后紧接 taken `BEQ`，延迟槽写入 `r3=0x55`，fall-through 对 `r4` 的写入必须被跳过，分支目标后 `MFLO/MFHI` 分别读出商 -4 和余数 -3，目标标记写入 `r7=0x66`。trace 中恰好出现 `r3/r5/r6/r7` 写回记录而没有 `r4` 写回记录，DUT 与独立参考 trace 全匹配。图 3 为该程序控制流的说明图。

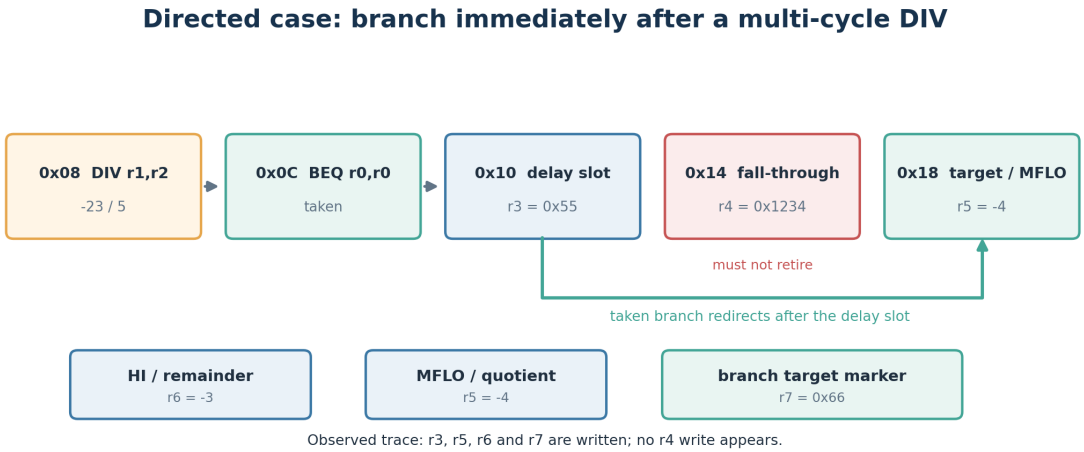


图 3: 除法占用 EX 时的分支等待、延迟槽和跳过路径定向检查

访存指令。访存控制由单一 load/store 信号扩为 12 位 one-hot `mem_op`，在一个字 RAM 接口上支持下表指令。地址按字索引访问 RAM，低地址位用于字节/半字通道选择；读数据由 MEM 阶段完成扩展或合并，写数据和 byte enable 在 EX 阶段按地址偏移生成。

类别	指令	实现要点
整字	`lw/sw`	读写 32 位字，`sw` 的四个 byte enable 全开。
字节	`lb/lbu/sb`	按地址低两位选 lane；`lb` 符号扩展，`lbu` 零扩展，`sb` 只使能目标 lane 并移位写数据。
半字	`lh/lhu/sh`	用地址位 1 选择半字；load 分别进行符号扩展或零扩展，store 使能对应两个 lane。
非对齐读	`lwl/lwr`	按低两位与旧 `rt` 合并；旧值来自经过 RAW 检测和前递的源操作数。
非对齐写	`swl/swr`	按低两位生成写掩码和移位后的写数据。

本实现沿用 32 位同步 data RAM，EX→MEM 总线增加有效地址和原 `rt` 值等信息。`lwl/lwr` 合并关系、`swl/swr` 四种偏移的掩码与数据排列已与独立 CPU132 reference 的执行级规则逐项核对。累计 Lab7 功能点回归通过并与 golden trace 匹配。

流水线接口与冒险处理

Lab6/Lab7 在原 forwarding 流水线接口上继续传递新增控制和数据。`DS_TO_ES_BUS_WD` 扩为 154 位，`ES_TO_MS_BUS_WD` 扩为 114 位。普通数据相关使用 EX/MEM/WB 前递；HI/LO 运算通过 EX 的 ready/allow 握手施加多周期背压；同步 load 数据只有在 RAM 返回后才可用。零号寄存器始终不接受写回。

普通 RAW 前递按最近写者优先选择 EX、MEM、WB、寄存器堆。若最近写者匹配但结果尚未有效，消费者等待，不越过它去取更旧结果。这个优先级对 ALU 输入、分支比较、`jr/jalr` 目标以及 store 写数据保持一致。同步 RAM 的时序没有改变，因此 load-use 仍可能停顿；这是等待数据到达的必要等待，不作为普通 ALU 前递失败。

实验方法与验收规则

累计课程映像的参考轨迹由独立 CPU132 参考环境生成；DUT 使用同一份 ROM MIF 运行 forwarding 模式。验收不仅检查 testbench 的功能点通过数，也检查参考 trace 是否打开、测试结束标记、PASS 标记、仿真退出码、使用的映像与源文件是否匹配，以及零号寄存器写回检查。每轮 Xvlog、Xelab、XSim 阶段的新鲜日志一并记录。

仿真周期数和停顿数直接引用该轮 `result.txt`。Lab6 与 Lab7 使用不同累计课程程序，因此二者周期数不能用来直接比较处理器性能。HI/LO testbench 和除法后分支程序作为补充覆盖；定向分支测试的课程功能点数为 0 是预期配置，它按自定义 trace 比较判定，不代表没有执行指令。

本轮按用户要求未启用 Vivado 波形捕获。图 1—3 是结构/程序流程说明图，图 4 是由验收结果绘制的功能点汇总图，均不是仿真波形。新增指令的真实信号级波形与逐图分析仍待人工采集；采集步骤和信号清单见 `docs/lab6_lab7/waveforms.md`。运行脚本、trace、日志及哈希用于复核可重复性。

实验结果

图 4 汇总 Lab6、Lab7 的累计课程功能点结果。详细数字和辅助检查见表 2。

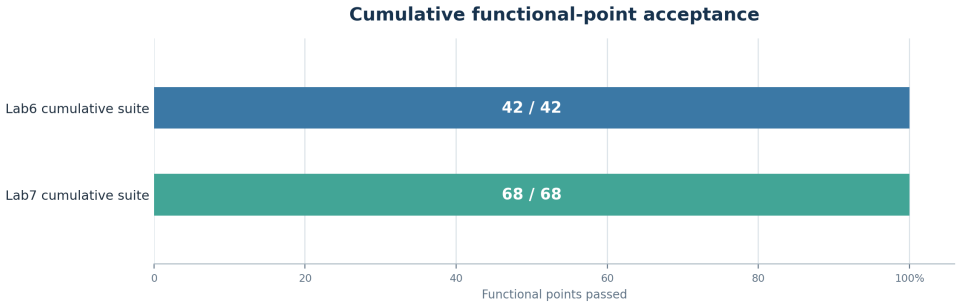


图 4: Lab6 与 Lab7 累计课程功能点通过率

指标	Lab6 累计	Lab7 累计
验收状态	PASS	PASS
课程功能点	42/42	68/68
流水线周期	109,375	166,226
流水线停顿周期	89	926
EX/MEM/WB 前递使用	35,013 / 8,320 / 12,007	49,970 / 12,229 / 15,454
新增指令前递	0 / 0 / 0 / 0	112 / 0 / 1,229 / 28
零号寄存器写回尝试	0（检查通过）	0（检查通过）
trace 与 ROM 映像	哈希匹配	哈希匹配
波形捕获	未启用	未启用

Lab6 累计 trace 共 66,612 条记录，SHA-256 为 `FB92799C...6986850CA1`；本轮课程 MIF SHA-256 为 `845D9793...C28E8F28`。Lab7 累计 trace 共 100,732 条记录，SHA-256 为 `23ADD75B...B22035991C3`；本轮课程 MIF SHA-256 为 `8EBF3140...B9B75BE`。完整哈希、原始绝对路径和运行起止时间保存在对应 `result.txt`。

EX/MEM/WB 前递计数仅在 ID 指令被 EX 接收时采样；每一级只要 `rs` 或 `rt` 任一源操作数使用该级就记一次，因此计数不是源操作数总数或停顿周期。新增指令计数也在发射时采样；store 基址与数据可能对同一条指令分别计数。完整 Lab7 累计程序的新增 store-base/store-data/conditional-branch/`jalr` 计数为 112/0/1,229/28。累计 ROM 没有触发新增 store-data 前递；独立短定向程序覆盖该依赖，trace 41/41 匹配，计数为 3/2/6/1。

两轮累计 DUT 均在预期功能点数上全部通过，参考 trace 与仿真镜像匹配；zero-register 检查均报告 0 次实际写回尝试。HI/LO 单元 testbench 和除法期间分支等待定向用例也通过。S2 历史结果曾在当时尚未实现 HI/LO 的 `mflo` 处通过 34/42；该历史状态不能代替当前完整 RTL 的 42/42 结果。

结果分析与局限

Lab6 与 Lab7 的累计功能点均达到本轮设定的完整通过条件。不同数据路径都获得独立证据：算术逻辑定向程序检查立即数、比较和可变移位；HI/LO testbench 检查多周期单元握手；自定义分支测试检查除法背压期间的延迟槽和重定向；累计课程 ROM 检查完整功能点并与独立参考退休 trace 逐项匹配。

当前验收证明的是指定课程映像和已记录定向场景下的仿真行为，不等同于形式化证明，也不包含板上实测、综合后时序收敛或所有非法输入组合。特别是除数为零没有列入 HI/LO testbench 覆盖；本报告不对该情况作正确性结论。本轮没有真实 Vivado 波形截图，新增指令波形及逐图分析属于尚待人工完成的报告事项；操作步骤和建议展示内容见 `docs/lab6_lab7/waveforms.md`。

总结

本次完成了 Lab6 算术逻辑与 HI/LO 乘除法扩展、Lab7 控制转移与延迟槽扩展，以及 12 类整字、字节、半字和非对齐访存操作。多周期单元通过 EX 级背压和完成消费握手避免重复执行；控制转移在 EX 停顿时等待，并保留一条延迟槽；访存操作按地址低位处理 lane、符号扩展和合并。

附录 A：复现命令与证据索引

从仓库根目录 `C:\Users\lenovo\Desktop\Projects\FPGA\lab` 的普通 Windows PowerShell 运行：

```
powershell.exe -NoLogo -NoProfile -ExecutionPolicy Bypass `
  -File .\scripts\verify.ps1 -Case lab6 -PipelineMode forwarding
powershell.exe -NoLogo -NoProfile -ExecutionPolicy Bypass `
  -File .\scripts\verify.ps1 -Case lab7 -PipelineMode forwarding
```

Vivado 不应从 AppContainer 受限 PowerShell 启动。仓库脚本会在 Vivado 启动前检查当前 token，并记录 host context。报告配套图可在仓库根目录用下列命令重新生成（需安装 Python 与 Matplotlib）：

```
python .\report\lab6-7\scripts\build_figures.py
```

表 1: 报告结论对应的原始证据

报告内容	仓库内证据
Lab3/Lab4 forwarding 回归	`results/lab3_lab4/lab3-20261026T162836Z/result.txt`；`results/lab3_lab4/lab4-20261026T163305Z/result.txt`。
Lab6 累计 DUT	`results/lab6_lab7/lab6-20261026T163659Z/result.txt`；同目录 Xvlog/Xelab/XSim 阶段日志。
Lab7 累计 DUT	`results/lab6_lab7/lab7-20261026T164130Z/result.txt`；同目录 Xvlog/Xelab/XSim 阶段日志。
新增指令前递定向用例	`results/lab6_lab7/lab7-20261026T170957Z/result.txt`；输入映像及 trace 位于 `results/lab6_lab7/directed/extended-forwarding/`。
HI/LO 单元 testbench	`results/lab6_lab7/hilo-unit-20261026T164656Z/result.txt` 与同目录 `console.log`。
除法期间分支定向用例	`results/lab6_lab7/directed/hilo-stall-branch/` 下程序映像及 golden trace；DUT 结果为 `results/lab3_lab4/lab3-20261022T065139Z/result.txt`。
算术逻辑定向用例	`results/lab6_lab7/directed/alu/` 和 `results/lab3_lab4/lab3-20261021T140840Z/`。
实现记录	`docs/lab6_lab7/implementation.md`。
完整验收与哈希	`docs/lab6_lab7/verification.md`。真实波形人工采集指南：`docs/lab6_lab7/waveforms.md`。
环境和运行入口	`docs/lab6_lab7/environment.md`。